UNIVERSITY OF ENGINEERING AND TECHNOLOGY

MARDAN

Name	**ASIM AYAZ**
Registration No#	**25MDBCS804**
Assignment	**Lab Assignment 7**
Submitted to	**Mam Alishba**
Semester	**2nd**
Date	**23-03-2026**

ASSIGNMENT 7

Q no 1:

Create a class testing having private data integer attributes as a & b and having a friend function that would initialize and display a & b values.

Source Code:

lab7_1.cpp > display(testing)

```cpp
#include<iostream>
using namespace std;

class testing{
    private:
    int a,b;
    public:

 friend void display(testing);

};
void display(testing obj){
    obj.a=5;
    obj.b=10;

    cout<<"a is :"<<obj.a<<endl;
    cout<<"b is :"<<obj.b<<endl;}
int main(){
    testing obj;
    display(obj);

    return 0;
}
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 (
 }
a is :5
b is :10
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no 2:

Implement the concept of friend function that will access the private data attributes of two classes, compare both the data and will return the greater data attribute value between the classes.

Source Code:

lab7_2.cpp > greaterData(test_one, test_two)

```cpp
#include<iostream>
using namespace std;
class test_two;
class test_one{
    private:
    int a;
    friend void greaterData(test_one, test_two);
    public:
    void getData(){
        cout<<"Enter a: "<<endl; cin>>a;
    }
};
class test_two{
     private:
     int b;
     friend void greaterData(test_one, test_two);
     public:
     void get(){
        cout<<"Enter b "<<endl; cin>>b;
     }
};
```

class test_two

```cpp
};
 void greaterData(test_one t1, test_two t2){
        if(t1.a>t2.b){
            cout<<"Class test_one is greater"<<endl;
        }
        else{
            cout<<"Class test_two is greater"<<endl;
        }
     }
int main() {
    test_one t1;
    test_two t2;
    t1.getData();
    t2.get();
    greaterData(t1, t2);
    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SPELL CHECKER

PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 OO
 }
Enter a:
12
Enter b
43
Class test_two is greater
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no 3:

Create a class named as containerClass having private data attributes as a & b, public data member functions such as getter() and putter() and a friend class containedClass. The getter() and putter() class will ask user to input & display private data attributes respectively. The containedClass would have setter() function that will display the containerClass private data attributes.

Source Code:

lab7_3.cpp > containedClass

```cpp
#include <iostream>
using namespace std;
class containedClass;
class containerClass {
private:
    int a, b;
public:
    void getter() {
        cout << "Enter values of a and b: ";
        cin >> a >> b;
    }
    void putter() {
        cout << "a = " << a << ", b = " << b << endl;
    }
    friend class containedClass;
};
```

```
class containedClass {
public:
    void setter(containerClass obj) {
        cout << "== Accessing the private data of containedClass using friend class:== \n";
        cout << "a = " << obj.a << ", b = " << obj.b << endl;
    }};
int main() {
    containerClass c;
    containedClass c1;
    c.getter();
    c.putter();
    c1.setter(c);
    return 0;
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SPELL CHECKER

PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 OOPS\OOPS LAB\
 }
Enter values of a and b: 12 23
a = 12, b = 23
== Accessing the private data of containedClass using friend class:==
a = 12, b = 23
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

ASSIGNMENT

Q no 4:

Imagine there is a function to operate on objects. The function will take objects as arguments, and will did addition operation on their private data using friend function.

Source Code:

lab7_4.cpp > add(Test, Test)

```cpp
#include <iostream>
using namespace std;
class Test {
private:
    int num;
public:
    void setValue(int n) {
        num = n;
    }
    friend int add(Test t1, Test t2);
};
int add(Test t1, Test t2) {
    return t1.num + t2.num;
}
int main() {
    Test t1, t2;
    t1.setValue(10);
    t2.setValue(20);
    cout << "Sum = " << add(t1, t2)<<endl;
    return 0;
}
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHEC

```
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semeste
 }
Sum = 30
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no 5:

Create two classes DM and DB which store the value of distances. DM stores distances in meters and centimeters and DB in feet and inches. Write a program that can read values for the class objects and add one object of DM with another object of DB. Use a friend function to carry out the addition operation. The object that stores the results may be a DM object or DB object, depending on the units in which the results are required. The display should be in the format of feet and inches or meters and centimeter depending on the object on display.

Source Code:

```
#include <iostream>
using namespace std;
class DB;
class DM {
private:
    int meters, centimeters;
public:
    void input() {
        cout << "Enter meters and centimeters: ";
        cin >> meters >> centimeters;
    }
    void display() {
        cout << "Distance(DM): " << meters << " m " << centimeters << " cm\n";
    }
    friend DM add(DM d, DB b);
```

```
class DB {
private:
    int feet, inches;
public:
    void input() {
        cout << "Enter int DB::inches ";
        cin >> feet >> inches;
    }
    void display() {
        cout << "Distance (DB): " << feet << " ft " << inches << " in\n";
    }
    friend DM add(DM d, DB b);
};
```

```
DM add(DM d, DB b) {
    int total_cm_dm = d.meters * 100 + d.centimeters;// here convert DM → cm
    int total_inches = b.feet * 12 + b.inches; //  here convert DB → cm
    int total_cm_db = total_inches * 2.54;
    int total_cm = total_cm_dm + total_cm_db;//here is add
    DM result;//here is convert back → meters + cm
    result.meters = total_cm / 100;
    result.centimeters = total_cm % 100;
    return result;
}
int main() {
    DM d;
    DB b;
    d.input();
    b.input();
    DM result = add(d, b);
    result.display();
    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SPELL CHECKER

PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1
 }
Enter meters and centimeters: 12 13
Enter feet and inches: 14 15
Distance(DM): 16 m 77 cm
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```